

BURSA TEKNİK ÜNİVERSİTESİ

Bilgisayar Mühendisliği Bölümü

Node.js ile Web Programlama — Dönem Projesi

Güvenli Sürüş ve Sürücü Davranışı Analizi Platformu

Senaryo 1: Mobil Güvenlik ve Davranış Analizi Platformu

GRUP 12

Numara	Ad Soyad
23360859019	Melike Rana Yozgatlı
23360859034	Selen Yakın
23360859739	Talha Korkmaz
22360859047	Kubilay İnanc

Haziran 2026

Özet

Bu çalışmada, akıllı telefonu araç içinde bir IoT cihazı gibi kullanan, sürücü davranışlarını gerçek zamanlı analiz eden tam yığın (full-stack) bir platform geliştirilmiştir. Telefonun ivmeölçer, jiroskop ve GPS sensörlerinden toplanan veriler Node.js/Express tabanlı bir sunucuya iletilmekte; eşik tabanlı bir anomali tespit motoru ani fren, ani hızlanma, sert dönüş, sarsıntı ve beklenmeyen hızlanma gibi riskli davranışları belirlemektedir. Sonuçlar MongoDB’de saklanmakta, Socket.io ile web paneline canlı olarak yayınlanmakta ve harita üzerinde görselleştirilmektedir. Sistem; Jest birim testleri (13/13 başarılı) ve gerçek bir iOS cihazı üzerinde yapılan uçtan uca testlerle doğrulanmıştır. Föyde tanımlı zorunlu modüllerin tamamı karşılanmış; ayrıca Swagger, otomatik testler ve çevrimdışı veri tamponlama gibi bonus özellikler eklenmiştir.

Anahtar kelimeler: sürücü davranışı, anomali tespiti, IoT, Node.js, gerçek zamanlı, mobil sensör.

Abstract

This project presents a full-stack platform that turns a smartphone into an in-vehicle IoT device for real-time driver behavior analysis. Accelerometer, gyroscope and GPS data are streamed to a Node.js/Express backend, where a threshold-based anomaly engine detects risky maneuvers such as hard braking, rapid acceleration, sharp turns, shaking and sudden speed changes. Results are stored in MongoDB, broadcast live to a React dashboard via Socket.io, and visualized on a map. The system was verified through Jest unit tests (13/13 passing) and end-to-end testing on a real iOS device. All mandatory modules defined in the project brief were implemented, along with bonus features such as Swagger documentation, automated tests and offline data buffering.

Keywords: driver behavior, anomaly detection, IoT, Node.js, real-time, mobile sensing.

1. Gereksinim Analizi

1.1 Problem Tanımı

Trafik kazalarının önemli bir kısmı agresif sürüş davranışlarından (ani fren, ani hızlanma, sert dönüş) kaynaklanmaktadır. Sürücülerin bu davranışlarının nesnel olarak ölçülmesi kişisel farkındalık yaratır, filo yöneticilerine veri sağlar ve sigorta primleri için adil bir taban oluşturur.

Akıllı telefonlar; ivmeölçer, jiroskop ve GPS gibi sürüş davranışı için gerekli tüm sensörleri barındırır. Bu projede telefon, araç içi bir IoT düğümü gibi kullanılarak yukarıdaki probleme yönelik tam yığın bir platform geliştirilmiştir.

1.2 Fonksiyonel Gereksinimler

#	Gereksinim	Föy Modülü
F1	Telefondan ivmeölçer + jiroskop + GPS verisi toplama	5.1
F2	Veriyi RESTful API ile Node.js backend'e iletme	5.1, 5.2
F3	JWT tabanlı kullanıcı girişi (admin + driver rolleri)	5.3
F4	Kullanıcı, cihaz, sürüş, sensör ve alarm kayıtlarını saklama	5.4
F5	Web panelinde gerçek zamanlı sensör akışı ve grafik	5.5
F6	Eşik tabanlı anomali tespiti	5.6
F7	Alarm gösterimi ve listeleme	5.7
F8	API dokümantasyonu, kurulum kılavuzu, proje raporu	5.8

1.3 Fonksiyonel Olmayan Gereksinimler

- **Performans:** Sensör verisi düşük gecikmeyle, ~500 ms aralıklarla iletilmeli.
- **Dayanıklılık:** İnternet kesintisi veriyi kaybettirmemeli (çevrimdışı tampon).
- **Güvenlik:** Parolalar bcrypt ile hashlenir, JWT secret ortam değişkeninde tutulur, hassas alanlar yanıtlardan filtrelenir.
- **Modülerlik:** Katmanlı mimari; controller/service/model ayrımı net.

2. Proje Tanımı

Sistem üç ana bileşenden oluşur:

- **Mobil İstemci (mobile/):** React Native (Expo). Sensör verisini toplar ve API'ye gönderir.
- **Backend (backend/):** Node.js + Express + MongoDB + Socket.io. API, kimlik doğrulama ve analiz.
- **Web Paneli (dashboard/):** React + Vite + Chart.js + Leaflet. İzleme, raporlama ve yönetim.

3. Kullanım Senaryosu

Sürücü

1. Sürücü mobil uygulamayı açar ve e-posta/parola ile giriş yapar.

2. "Sürüş Başlat" butonuna basar; sensörler aktive olur ve backend'de bir sürüş kaydı açılır.
3. Yola çıkar; telefon ~500 ms aralıkla sensör verisini API'ye gönderir.
4. Ani fren gibi bir davranışta anomali algoritması alarm üretir; web paneline anlık bildirim düşer.
5. Sürüş bitince "Durdur"a basar; kayıt kapatılır ve risk skoru hesaplanır.

Yönetici (Admin)

1. Admin web paneline giriş yapar.
2. Tüm sürücülerin sürüşlerini, alarmlarını ve cihazlarını görüntüler.
3. Riskli sürücülerini tespit eder ve alarm geçmişlerini inceler.

4. Sistem Mimarisi

Sistem dört ana bileşenden oluşan modüler, katmanlı bir mimariye sahiptir. Mobil istemci sensör verisini HTTP üzerinden backend'e gönderir; backend veriyi doğrular, anomali motorundan geçirir ve MongoDB'ye kaydeder. Tespit edilen olaylar ve canlı sensör verisi Socket.io ile web paneline anlık olarak yayınlanır.

Katmanlar

- **Sunum katmanı:** Mobil uygulama ve web paneli (kullanıcı arayüzü).
- **API katmanı:** Express route ve controller'ları; HTTP isteklerini karşılar.
- **İş mantığı katmanı:** Servisler; anomali tespiti ve alarm üretimi.
- **Veri katmanı:** Mongoose şemaları ve veritabanı sorguları.
- **Gerçek zamanlı katman:** Socket.io ile canlı veri yayını.

5. Veri Modeli

Veriler beş ana Mongoose koleksiyonunda tutulur. Varlıklar arasındaki ilişkiler aşağıdaki gibidir:

Varlık	Açıklama	İlişki
User	Kullanıcı bilgileri, rol (admin/driver), bcrypt parola	—
Device	Sürücünün telefonu; UUID, model, OS, son görülme	User 1—N
Trip	Sürüş oturumu; süre, mesafe, ort. hız, alarm sayısı, risk skoru	User 1—N
SensorData	Zaman damgalı sensör örnekleri (en yoğun koleksiyon, indeksli)	Trip 1—N
Alarm	Anomali kaydı; tür, şiddet, ölçülen değer, eşik, konum	Trip 1—N

6. Kullanılan Teknolojiler

Katman	Teknoloji	Gerekçe
Backend	Node.js, Express.js	Dersin temeli, olgun ekosistem
Veritabanı	MongoDB, Mongoose	Zaman serisi veri için esnek şema

Katman	Teknoloji	Gerekçe
Gerçek zamanlı	Socket.io	Düşük gecikmeli canlı yayın, oda yapısı
Kimlik doğrulama	JWT, bcrypt	Stateless oturum, güvenli parola
Mobil	React Native, Expo (SDK 54)	Hızlı geliştirme, hazır sensör API'leri
Web paneli	React, Vite	Modern SPA, hızlı geliştirme
Görselleştirme	Chart.js, Leaflet	Zaman serisi grafiği ve harita
Dokümantasyon	Swagger / OpenAPI	İnteraktif API dokümantasyonu
Test	Jest	Detektör birim testleri
Sürüm kontrol	Git, GitHub (monorepo)	Ekip çalışması, tek repo

7. Gerçekleştirilen Modüller

7.1 Mobil Veri Toplama (Föy 5.1)

- Expo Sensors ile ivmeölçer ve jiroskop, Expo Location ile GPS verisi toplanır.
- Yerçekimi kompanzasyonu:** Ham ivme yerçekimini içerir; low-pass filtre ile yerçekimi çıkarılarak gerçek hareket (lineer ivme) elde edilir.
- Saniyede 2 örnek; ~3 sn'lik gruplar (batch) halinde gönderim.
- Çevrimdışı tampon (AsyncStorage) ile internet kesintilerine dayanıklılık.

7.2 Backend (Föy 5.2)

- Express MVC mimarisi; merkezi hata yönetimi, CORS, loglama (morgan/winston).

7.3 Kimlik Doğrulama ve Rol (Föy 5.3)

- JWT doğrulama middleware'i ve rol bazlı yetkilendirme (admin / driver).

7.4 Veritabanı (Föy 5.4)

- 5 Mongoose şeması, performans indeksleri ve özet alanları.

7.5 Gerçek Zamanlı İzleme Paneli (Föy 5.5)

- Socket.io ile canlı veri akışı, Chart.js zaman serisi grafiği, Leaflet ile rota haritası.

8. Test Süreci

8.1 Birim Testleri (Jest)

Anomali detektörleri bilinen senaryolarla test edilmiş ve 13 testin tamamı başarıyla geçmiştir (normal sürüşte alarm üretilmemesi, eşik aşımında doğru tür/şiddette alarm, sınır durumları).

8.2 Uçtan Uca Entegrasyon Testi

- Backend uç noktaları (health, login/JWT, sensor-data, trips, alarms) hatasız çalışmaktadır.
- Anomali motoru API üzerinden canlı alarm üretmektedir (ör. -5 m/s^2 fren $\rightarrow$ ani fren alarmı).
- Socket.io canlı veri akışı ve alarm bildirimi (toast) doğrulanmıştır.

8.3 Gerçek Cihaz Testi (iOS)

Mobil uygulama gerçek bir iPhone üzerinde Expo Go ile çalıştırılmış; telefonun gerçek ivmeölçer, jiroskop ve GPS verisi backend'e iletilmiş, gerçek hareketlerle anomaliler tetiklenmiş ve web panelinde canlı görüntülenmiştir. Bu test, sistemin yalnızca simülasyonla değil fiziksel sensörlerle de uçtan uca çalıştığını doğrular.

8.4 API Testi

Swagger UI (/api-docs) üzerinden tüm uç noktalar interaktif olarak test edilebilir.

9. Karşılaşılan Zorluklar ve Kısıtlar

9.1 Çözülen Mühendislik Zorlukları

1. **Sensör gürültüsü:** Ham ivme her titreşimde alarm üretiyordu; hareketli ortalama + debounce ile yanlış alarmlar azaltıldı.
2. **Yerçekimi etkisi:** Dik tutulan telefonda bir eksen sürekli $\sim -9.8 \text{ m/s}^2$ okuyordu; low-pass filtre ile lineer ivme elde edildi.
3. **Canlı veri akışı:** Mobilde stale-closure nedeniyle veri yalnızca sürüş bitince gidiyordu; React ref ile anlık akış sağlandı.
4. **Araç zinciri uyumu:** Node.js 24 ile eski Expo SDK uyumsuzdu; proje Expo SDK 54'e hizalanarak (iPhone uyumu dahil) çözüldü.
5. **Geçersiz GPS hızı:** iOS hız yokken -1 döndürüyordu; negatif/geçersiz değerler 0 olarak işlendi.

9.2 Mevcut Kısıtlar

1. Eşik değerleri literatürden alındı; geniş ölçekli gerçek araç testleriyle kalibrasyon idealdir.
2. Telefon yönelimi ivme eksenlerini etkiler; üretimde bir kalibrasyon adımı gerekir.
3. GPS hassasiyeti nedeniyle "güzergâh dışına çıkma" detektörü MVP kapsamına alınmadı.
4. Sürekli sensör + GPS + WebSocket batarya tüketir; üretimde düşük güç modu gerekir.
5. SensorData hızlı büyür; üretimde MongoDB Time Series veya bölümlleme önerilir.

10. Sonuç

Proje föyünün tüm zorunlu modülleri (5.1–5.8) karşılanmıştır. Bonus kapsamında Swagger/OpenAPI entegrasyonu, Jest birim testleri, Socket.io ile gelişmiş gerçek zamanlı yapı ve çevrimdışı veri tamponlama eklenmiştir.

Sistem; sürücü hareketlerini saniyede 2 örnek hızında izleyen, anomalileri ~3 saniye içinde tespit edip yayınlayan, gerçek bir iOS cihazında uçtan uca doğrulanmış ve ekipçe geliştirilmesi kolay çalışan bir prototip olarak teslim edilmektedir. Projenin kaynak kodu: github.com/MandallF/safe-drive-platform